

docker_class_10(53)

basic intro of swarm:

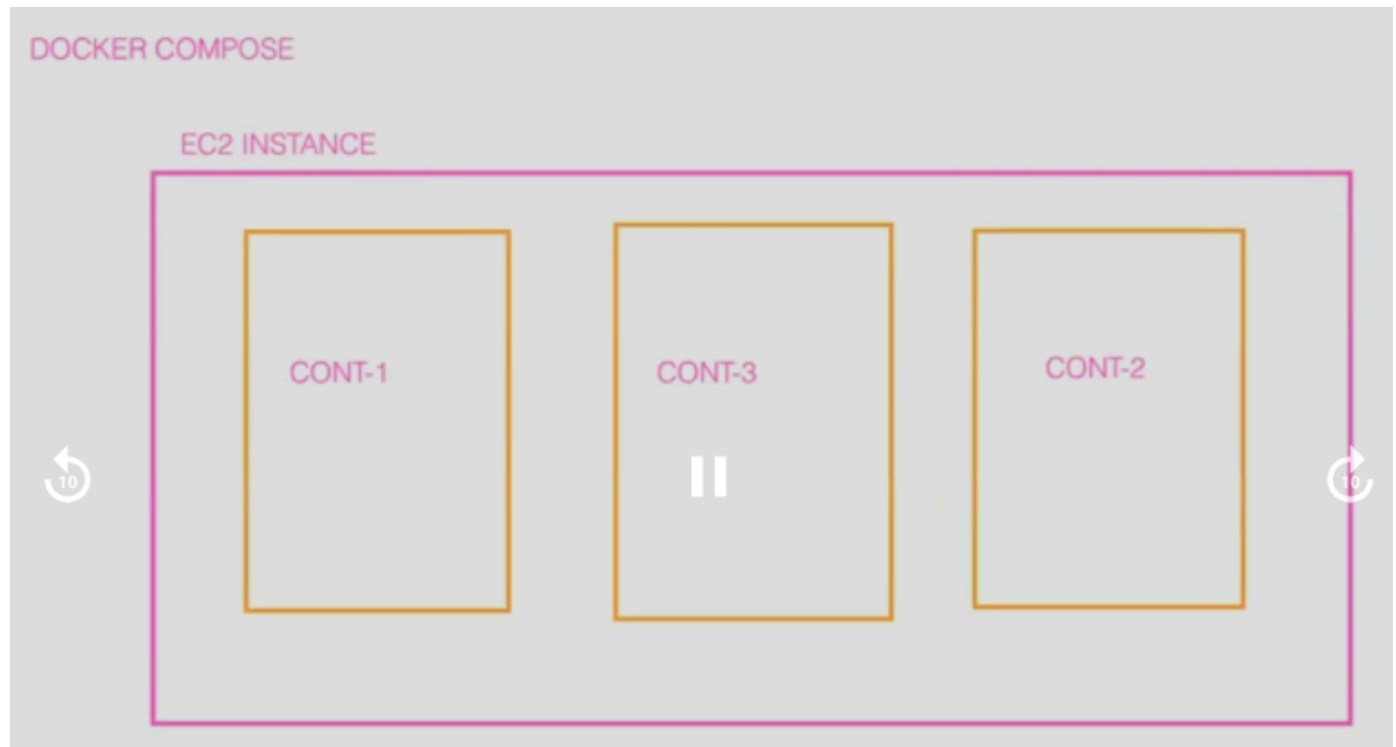
means multiple containers on multiple servers at a time

this we have discussed previous class this class will discuss docker-compose

docker swarm opposite to docker-compose

docker-compose means one server will have multiple container i.e app cont db cont

at a time if we want multiple container in one single server we use docker compose



Q1: you can ask starting also we did are we did individually we created or at a time we did individually if we want to at a time all containers we use docker compose

How we create means yaml file docker-compose.yaml file

DOCKER COMPOSE:

- It is a tool used to build, run and ship multiple containers for an application.
- It is used to create multiple containers in a single host.
- It uses a YAML file to manage multiple containers as a single service.
- The Compose file provides a way to document and configure all of the application's service dependencies (databases, queues, caches, web service APIs, etc).

COMMANDS:

- Start all services: `docker-compose up`
- Stop all services: `docker-compose down`
- Run Docker Compose in detached mode: `docker-compose up -d`

- List running containers: `docker ps`

Key Points from Your Image

The image covers the basics of Docker Compose V1 (Legacy):

- **What it is:** A tool to manage multi-container applications (Services, Networks, and Volumes).
- **Default path is :** Usually named `./docker-compose.yml` or `compose.yml`.
- **Service Definition:** Configures the behavior and environment for each container in the service.

IN one file only we will maintain cont1 and cont2 with app and db with ports and iamges etc if we excute all should excute.

crating new ec2 server install docker

Docker-compose Installation Commands

```
sudo apt-get update
sudo apt-get install docker-compose-plugin
mkdir -p ~/.docker/cli-plugins/
curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86\_64 -o
~/.docker/cli-plugins/docker-compose
chmod +x ~/.docker/cli-plugins/docker-compose
docker compose version
```

or

```
sudo curl -L "https://github.com/docker/compose/releases/download/1.29.1/docker-compose-\$\(uname -s\)-\$\(uname -m\)" -o /usr/local/bin/docker-compose
ls /usr/local/bin/
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose version
```

Verify Docker Engine & Compose Link and Testing with a sample project

```
nano docker-compose.yml
```

services:

hello-world:

image: hello-world

```
docker compose up
```

If you see "Hello from Docker!", your installation is 100% functional.

will create one samplefile but before going to that we can give only this 4 names only

docker-compose.yaml
docker-compose.yml
compose.yml
compose.yaml

1. single service

vim docker-compose.yaml

```
version: "3"
```

```
services:
```

```
  flm:
```

```
    image: nginx
```

```
    ports:
```

```
      - "8081:80"
```

Run this file

docker compose up -d

how to check is it service is created?? docker service ls it wont show due to it is not swarm

docker ps

```
harish@aold2402006:~$ docker ps
```

```
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

```
455a2eb78ef9 nginx "/docker-entrypoint..." 6 minutes ago Up 6 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp test-flm-1
```

here one container will create

here if we want to down the service just do

docker-compose down

automatically it will stop and removed and container also removed

Docker Compose file and their descriptions:

- **version** – Specifies the version of the Compose file format.
- **services** – Defines the individual containers (services) that make up your application.
- **networks** – Allows you to define the isolated networking setup for your containers to communicate.
- **volumes** – Defines persistent data storage used by your application.
- **configs** – Lets you add external configuration files to your containers, keeping them generic and portable

2. multiple services

version: "3"

services:

pubg:

image: shaikmustafa/cycle

ports:

- "8081:80"

swiggy:

image: shaikmustafa/dm

ports:

- "8082:80"

zomato:

image: shaikmustafa/mygame

ports:

- "8083:80"

Run the app:

docker compose up -d

Compose will create **3 containers**:

harish@aold2402006:~/test\$ docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

79004c93342d	shaikmustafa/mygame	"/docker-entrypoint...."	40 seconds ago	Up	39 seconds	
--------------	---------------------	--------------------------	----------------	----	------------	--

0.0.0.0:8083->80/tcp, [::]:8083->80/tcp	test-zomato-1
---	---------------

240a94435517	shaikmustafa/cycle	"/docker-entrypoint...."	40 seconds ago	Up	39 seconds	
--------------	--------------------	--------------------------	----------------	----	------------	--

0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	test-pubg-1
---	-------------

88f7a5a242a9	shaikmustafa/dm	"/docker-entrypoint...."	40 seconds ago	Up	39 seconds	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
test-swiggy-1						

3. advance

vi docker-compose.yaml

mkdir movie train

cd movie

vim index.html

<h1>This is movie ticket service</h1>

vi Dockerfile

```
FROM nginx
COPY . /usr/share/nginx/html
```

Now repeat for **train service**:

```
cd ../train
vim index.html
```

```
<h1>This is train ticket service</h1>
```

```
vi Dockerfile
```

```
FROM nginx
COPY . /usr/share/nginx/html
```

Build images

If we build both services, two images will be created.

```
docker build -t image1 movie/
docker build -t image2 train/
```

We name them:

- movie → image1
- train → image2

Verify images

```
docker images
```

If we define both `image1` and `image2` in a `docker-compose.yml` file, then running: will create **two separate containers**, one for each service.

```
vi docker-compose.yml
```

```
version: "3"
```

```
services:
```

```
  movie:
```

```
    image: image1
```

```
    ports:
```

```
      - "8084:80"
```

```
  train:
```

```
    image: image2
```

ports:

- "8085:80"

docker-compose up

its working now now devloper did changes movie tickets

harish@aold2402006:~/test\$ docker ps

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

83f645685974 image1 "/docker-entripoint...." 24 hours ago Up 8 seconds 0.0.0.0:8084->80/tcp,
[::]:8084->80/tcp test_movie_1

06823bacaf0a image2 "/docker-entripoint...." 24 hours ago Up 8 seconds 0.0.0.0:8085->80/tcp,
[::]:8085->80/tcp test_train_1

cd movie

vim index.html

<h1>This is movie ticket service</h1>

<h2>welcome to paytm service</h2>

But Docker is running a **previous image build**, not your latest file.

Docker images are *snapshots*, not live code

is the code reflect in app ans is No

we have to rebuild image properly

docker build -t image1 movie/

force recreate containers

docker compose up -d --build

✓ Container 83f645685974_test_movie_1 Recreated this way it will show

when code is changes after that needs to do rebuild the image and docker compose up we have to do

Basic Flow

DEV → CODE → CHANGES

DOCKER IMAGE BUILD: Once the code is ready, you manually build the Docker image using a Dockerfile.

DOCKER COMPOSE UP: You then use Docker Compose to start the application based on those builds.

Multiple Services Scenario

10 SERVICES → CODE: Imagine your application is split into 10 different microservices (e.g., database, frontend, backend, cache, etc.), each with its own code.

10 IMAGES: Instead of manually building 10 separate images one by one, Docker Compose handles the complexity.

COMPOSE UP: With a single command, `docker compose up`, you can build all 10 images and start all 10 services simultaneously.

The second half of the image highlights why Compose is so powerful when dealing with complex applications:

what i will do i will show below

services:

movie:

build: ./movie/

ports:

- "8084:80"

train:

build: ./train/

ports:

- "8085:80"

in build we will mention path of service

now i will two service in changes train and movie

For the Movie service:

nano /home/harish/test/movie/Dockerfile

FROM nginx:alpine

RUN echo "<h1>Welcome to the Movie Service</h1>" > /usr/share/nginx/html/index.html

****For the Train service:**

****nano /home/harish/test/train/Dockerfile**

FROM nginx:alpine

RUN echo "<h1>Welcome to the Train Service</h1>" > /usr/share/nginx/html/index.html

Now, go back to your main test folder and run the build again:

if i do like i dont need to do build individually just we have to give **docker compose build** or ****docker compose up --build -d**

Verify

curl localhost:8084

curl localhost:8085

its working

again Adding Data to the Code (Static Data into movie)

```
cd ~/test/movie$ vi Dockerfile
```

```
FROM nginx:alpine
```

```
RUN echo "<h1>Welcome to the Movie Service</h1> <h2>hey bro devops guru</h2>" >
/usr/share/nginx/html/index.html
```

Output:

```
harish@aold2402006:~/test/movie$ curl http://localhost:8084/
```

```
<h1>Welcome to the Movie Service</h1> <h2>hey bro devops guru</h2>
```

steps:

1. compose file keep build needs to keep as well as path

2. docker compose up --build -d the build path it will build while run docker compose up --build -d
build vs image we understood why we use build

`docker compose config` → shows *final interpreted result

`docker compose images` -> **It shows which images your containers are using**, along with details like:

`docker compose stop` -> This command **stops running containers** created by your Compose file — but **does not remove them**.

`docker compose down` -> **Stops and removes** containers + network

`docker compose up` -> Creates + starts containers

`stop` = pause

`down` = destroy + clean restart

*****Apply it to your case

- Your `movie` and `train` folders → code
- `docker build` → creates images
- `docker compose up` → starts containers using those images

we can do pasue and unpass also i will tell by container way

if we found any bug what we will do means we will stop and fix and then start instead of that we can do pause if we make pasue to the container it wont happen updates in the container

it will not get any updates from outside then later fix the bug we can do unpause If we find a bug, we usually **stop the container, fix the issue, rebuild the image, and start again**.

Using **pause** is not a replacement for this.

What `pause` actually does

```
docker pause <container>
```

What `pause` actually does

- Freezes all processes inside the container

- CPU usage → stops
- Container state → unchanged
- No code changes are applied

`docker unpause <container>`

Resumes exactly where it left off

- `docker compose stop` → safely stops the app
- `docker compose start` → starts the same app again
- `docker pause` → freezes the app exactly as-is

Correct workflow

If you find a bug:

`docker compose down`

fix code

`docker compose up -d --build`

```
harish@aold2402006:~/test$ docker compose pause
```

```
WARN[0000] /home/harish/test/compose.yaml: the attribute `version` is obsolete, it will be ignored,
please remove it to avoid potential confusion
```

```
[+] pause 2/2
```

```
✓ Container test-movie-1 Paused 0.0s
```

```
✓ Container test-train-1 Paused 0.0s
```

```
harish@aold2402006:~/test$ curl http://localhost:8084/
```

```
^C
```

```
harish@aold2402006:~/test$ curl http://localhost:8084
```

If we pause a container, it does not stop; it freezes all processes inside it.

The application will not respond during this time.

When we unpause, the application resumes from the same state and continues working.

Pause = freeze everything, Unpause = continue from where it stopped

pause inside networks and files it will works

To check logs of compose i.e: `docker compose logs`

i want to see only compose file containers: `docker compose ps`

To excute compose file: `docker compose up -d`

to build the docker files: `docker compose build`

To delete all services: `docker compose down`

To stop all containers: `docker compose start`

To pause all container: `docker compose pause`

TO unpause all containers: docker compose unpause

TO get list of containers: docker compose ps

To get list of images: docker compose images

To get config details of compose file: docker-compose config

vi compose.yaml

services:

one:

image: nginx

ports:

- "8081:80"

volumes:

- volume-one:/usr/share/nginx/html # Source:Destination

networks:

- network-one

two:

image: httpd

ports:

- "8082:80"

volumes:

- volume-two:/usr/local/apache2/htdocs # Source:Destination

networks:

- network-two

networks:

network-one:

driver: bridge

network-two:

driver: bridge

volumes: # You MUST declare named volumes here

volume-one:

volume-two:

docker compose up

docker ps

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

435cbb418842 httpd "httpd-foreground" 7 minutes ago Up 7 minutes 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp test-two-1

c46a230cdd08 nginx "/docker-entrypoint...." 7 minutes ago Up 7 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp test-one-1

harish@aold2402006:~/test\$ docker volume ls |grep test

local test_volume-one

```
local test_volume-two
harish@aold2402006:~/test$
```

```
harish@aold2402006:~/test$ docker network ls | grep test
1bbeb5946366 test_default bridge local
329536a6140f test_network-one bridge local
ad5abcd4a641 test_network-two bridge local
```

here volumes and networks and container it will created what ever we given composefile

DATABASE CONTAINER

IN REALTIME DATABASES WILL CREATE CONTAINER so manam kuda cheyochu that using volume they will create containers

we can create database container using volume
to create volume : docker volume create database
to check : docker volume ls

using this volume we will create one container

Note: container images also it should be mysql
databases sambandinchi use chesukoni create container
to create container database

```
docker run -d --name database-cont -v database:/database -p 3306:3306 -e
MYSQL_ROOT_PASSWORD=root mysql/mysql-server:5.7
```

chatgpt suggested:

```
docker run -d --name database-cont -v database:/var/lib/mysql -p 3306:3306 -e
MYSQL_ROOT_PASSWORD=root mysql/mysql-server:5.7
```

database have username and password

-e environment it is used for env using this we can access in container whereas arg we cannot access

-d used means we can enter via exec

to login container

```
docker exec -it database-cont /bin/bash
```

to login mysql give below command username we didnot provided so username but it will root as a user

```
bash-4.2# mysql -uroot -proot
```

```
mysql>SHOW DATABASES;
```

so here backend teams will work here

what we will do means

1. write docker file and create scripts ki build docker file chesi image took and update the image